- The exciting potential of including third-party providers into this type of testing, with people we speak to often saying that third-party providers are a key challenge for them

CHALLENGES OF PDCT

With PDCT, consumers who use adapters to generate contracts from mocks lose out on the loose matching functionality. This functionality allows for more flexible data matching, which removes the friction of static response data, for example, date fields.

PDCT also offers much less assurance as the verification status is only a flag from the provider stating the test status for the OpenAPI specification interaction, instead of the deep comparison like with CDCT. In addition, PDCT doesn't provide insight into consumer usage of the API.

Using provider integration tools such as Postman requires spinning up the application independently of the tests themselves, making it less efficient than running the tests locally, like with the native Pact approach. In addition, if the tests live separately from the code, it may be more difficult to debug any problems that occur.

3.8 Communication types supported by contract testing

In chapter 1, we explained contract testing alongside REST APIs. However, there are other interfaces that the Pact framework can support. GraphQL and event-driven systems have gained popularity in recent years as well. Both still have interfaces between services that could require contract tests to verify their relationship.

3.8.1 GraphQL

GraphQL has an interface very similar to REST but with a query language to request and update specific pieces of data. This design allows consumers to be much more flexible with what they request. In the example of a web